

i. Co je to JavaScript

JavaScript je jednoduchý skriptovací jazyk, který se zapisuje přímo do HTML kódu stránky mezi speciální značky a je interpretován na straně klienta. Znamená to, že pokud je stránka doplněná o nějaký javascriptový program, tak ke spuštění programu nedojde na serveru, kde jsou stránky uloženy, ale přímo na počítači, z něhož je stránka prohlížena. Proto není pro jejich tvorbu zapotřebí žádného speciálního nástroje, či překladače a zdrojové kódy jsou všem přístupné. To má samozřejmě své výhody i nevýhody.

JavaScript je objektově orientovaný jazyk a jeho syntaxe vychází z jazyka C++.

JavaScript má také svá omezení, z nichž nejpodstatnější souvisí s prací se soubory.

Vzhledem k tomu, že JavaScript je spouštěn na počítači klienta, tak z pochopitelných důvodů nedokáže uchovávat, měnit nebo mazat soubory, které se nacházejí na serveru.

1 Zařazení JavaScriptu do stránky HTML

1.1 Do HTML stránky

```
<SCRIPT LANGUAGE="JavaScript">
<!--
document.write("ahoj světe");
// -->
</SCRIPT>
```

Do hlavičky – spustí se až voláním

Do těla (body) – spustí se hned

Pravidla syntaxe

- Příkazy jsou odděleny středníkem.
- Řetězce se dávají do uvozovek.

1.2 Vložení JavaScriptu do externího souboru.

Obsah externího souboru s názvem **soubor.js** je

```
document.write("Toto je text v externím souboru ");
```

Pak se externí soubor zavolá

```
<SCRIPT LANGUAGE="JavaScript" SRC="soubor.js"></SCRIPT>
```

1.3 In-line zápis

In-line znamená "v řádku" a je to třetí způsob začlenění JavaScriptu do HTML stránek. Nevyužívá tag `<script>`, ale zapisuje se jako atribut jiného tagu. Většinou reaguje na nějakou uživatelskou událost. Raději ukážu příklad:

```
<a href="http://www.meuslany.cz" onmouseover="alert('Na úřadě je myš!!!!')">Městský úřad Slaný</a>
```

Komentáře

```
// toto je komentář
```

```
/*
```

```
I zde je komentář
```

```
*/
```

Datové typy (hodnoty)

JavaScript obsahuje následující typy hodnot

- čísla, jako např. 42 nebo 3.14159
- logické (Boolean) hodnoty, true anebo false
- řetězce (string) , jako např. "Ahoj"
- null, speciální klíčové slovo věnované prázdné hodnotě

ii. Jména proměnných

JavaScript identifikátor nebo jméno musí začínat písmenem či podtržítkem ("_"); následující znaky mohou být taktéž číslice (0-9). Písmena jsou znaky od "A" do "Z" (velká) a též od "a" do "z" (malá). JavaScript je citlivý na malá a velká písmena (case-sensitive).

iii. Celá čísla (integers)

Celá čísla mohou být vyjádřena v desítkové, hexadecimální či oktalové soustavě. Číslo v desítkové soustavě se skládá z posloupnosti číslic bez počáteční nuly.

iv. Reálná čísla

Reálná čísla mají tyto části: celé číslo, desetina tečka ("."), zlomek (jiné celé číslo), exponent, přípona typu. Exponent je vyjádřen pomocí "e" či "E" následovaný celým číslem, které může mít znaménko ("+" nebo "-"). Reálná čísla musí mít aspoň jedno číslo plus desetinou tečku anebo "e" (či "E").

Příklady

3.1415

-3.1E12

.1e12

2E-12

V JavaScriptu je maximální číslo $\pm 1,797 \times 10^{308}$ a minimální číslo je $\pm 2,2 \times 10^{-308}$.

v. Typ Boolean

Typ Boolean má dva literály : **true** a **false**.

vi. Řetězcové proměnné

Řetězcový literál je nula a více znaků uzavřených v uvozovkách (") nebo v apostrofech ('). Řetězec musí být oddělen znaky stejného typu - tj. buď jsou oba uvozovky, či oba apostrofy. Příklady řetězcových literálů :

```
"blah"  
'blah'  
"1234"  
"one line \n another line"
```

Speciální znaky

Můžeme používat tyto speciální znaky v JavaScriptových řetězcových literálech:

\b backspace.
\f form feed.
\n new line character.
\r carriage return.
\t tab character.
\" Znak "

Do řetězce můžeme vložit uvozovky tím, že je bude předcházet zpětné lomítko (backslash). Toto je známo jako escapování uvozovek.

Příklad

```
document.write("toto je znak \"")  
Výsledkem tohoto je : toto je znak "
```

b. Výrazy

Operátory přiřazení (=, +=, -=, *=, /=)

Přiřazovací operátor přiřadí hodnotu levému operandu na základě hodnoty pravého operandu. Základním přiřazovacím operátorem je rovnítko (=), které přiřadí hodnotu pravého operandu levému (x = y).

Ostatní operátory jsou zkratky pro standardní aritmetické operace:

```
x += y znamená x = x + y  
x -= y znamená x = x - y  
x *= y znamená x = x * y  
x /= y znamená x = x / y  
x %= y znamená x = x % y
```

c. Aritmetické operátory

Aritmetické operátory berou číselné hodnoty a vracejí jednoduchou číselnou hodnotu.

i. Standardní aritmetické operátory

Standardní aritmetické operátory jsou sčítání (+), odčítání (-), násobení (*), dělení (/) a celočíselné dělení (%).

Zvětšení - increment (++)

Při postfixu vrátí hodnotu před zvětšením, při prefixu po zvětšení.

ii. Příklad 1

Jestliže x je 3, pak příkaz	y = x++	zvětší x na 4 a nastaví y na 3.
Jestliže x je 3, pak příkaz	y = ++x	zvětší x na 4 a nastaví y na 4.
Zmenšení - decrement (--)		

Při postfixu vrátí hodnotu před zmenšením, při prefixu po zmenšení.

iii. Příklad 2

Jestliže x je 3, pak příkaz `y = x--` zmenší x na 2 a nastaví y na 3.

Jestliže x je 3, pak příkaz `y = --x` zmenší x na 2 a nastaví y na 2.

Unární negace (-)

Operátor unární negace musí předcházet daný operand. Například

`x = -x`

Vezme zápornou hodnotu x. Jestliže x bylo 3, stalo se -3.

Řetězcové operátory

Pro řetězec je definován operátor (+), neboli spojení dvou řetězců do jednoho. Například

`"můj " + "řetězec"` vrátí řetězec `"můj řetězec"`.

iv. Priorita operátoru

Priorita operátoru určuje jejich pořadí při vyhodnocování výrazu. Toto můžeme změnit použitím závorek.

Priorita od nejnižší do nejvyšší

čárka ,

přiřazení `= += -= *= /= %= <<= >>= >>>= &= ^= |=`

podmínka `?:`

logické-or `||`

logické-and `&&`

bitový-or `|`

bitový-xor `^`

bitový-and `&`

rovnost `== !=`

porovnání `< <= > >=`

posun `<< >> >>>`

sčítání/odčítání `+` `-`

násobení/dělení `*` `/` `%`

negace/zvětšení/zmenšení `!` `~` `-` `++` `--`

call, member `() []` .

v. Pole

Definice

`Pole=new array();`

`Pole[0]="ahoj";`

`Pole[1]="světe";`

Definice 2

`Pole=new array("ahoj", "světe");`

`Document.write(Pole[1]);`

Příklad

`<SCRIPT LANGUAGE="JavaScript">`

`<!--`

`den=new Array("pondělí", "úterý", "středa", "čtvrtek", "pátek", "sobota", "neděle");`

`denes=3;`

`document.write("Dnes je: "+den[denes]+"<BR>");`

`document.write("Zítra bude: "+den[denes+1]+"<BR>");`

```
document.write("Předevčírem byl: "+den[dnes-2]+"<BR>");
document.write("Za 3 dny bude: "+den[dnes+3]+"<BR>");
// -->
</SCRIPT>
```

vi. Spojování polí - concat

```
cele=pole1.concat(pole2);
Join
```

vii. Převod prvků pole do řetězce odděleného oddělovačem - join

```
a=pole1.join(";");
```

Příklad

```
pole1=new Array(1,2,3);
```

```
a=pole1.join(".");
```

```
document.write(a);
```

Výsledkem je výpis všech prvků pole oddělených tečkou - 1.2.3.

Délka pole - length

Pole1.length

Přidání prvku do pole

split

```
pole=split(cookie,“;“)
```

viii. Odebrání prvku na konci pole

Příklad

```
Pole.length=3
```

```
pole2=new Array("čtvrtek","pátek");
```

```
pole2.length=1;
```

```
document.write(" je: "+pole2[0]+"<BR>");
```

```
document.write(" je: "+pole2[1]+"<BR>");
```

ix. Setřídění pole - Sort

Příkaz sort slouží k setřídění pole.

```
nove=pole1.sort();
```

x. Reverzní pole

Pro vytvoření reverzního pole se používá příkaz **reverse**.

```
re=pole1.reverse();
```

xi. Slice

Slice – vybere prvky pole od do

```
nove=pole1.slice(4,5);
```

1. Příklad - spojí obsah všech polí

```
<HTML>
```

```

<HEAD>
<TITLE>Ukázka polí v praxi - den v týdnu</TITLE>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!--
var a= [1,2,3,4,5,6,7,8];
for (i=0; i<a.length;i+=1)
    document.write(" je: "+a[i]+"<BR>");
a.splice(1,1)
for (i=0; i<a.length;i+=1)
    document.write(" je: "+a[i]+"<BR>");
// -->
</SCRIPT>
</BODY>
</HTML>

```

xii. Podmínky

Podmínkový výraz

Podmínkový výraz může podle podmínky nabýt jednu ze dvou hodnot. Syntaxe je

$C = (\text{výraz}) ? a : b$

Jestliže je podmínka pravdivá (true), výraz získá hodnotu a, jinak získá hodnotu b.

Příklad

vekova_kategorie = (vek>=18) ? "dospělý":"dítě";

Podmínka if

If (výraz) proved' A else proved' B

Příklad

vek=19;

if (vek>=18) vekova_kategorie="dospělý";

else vekova_kategorie="dítě";

document.write("Máte "+vek+" rok(ů),
");

document.write("patříte do kategorie "+vekova_kategorie+".");

Více příkazů je umístěno mezi symboly { }.

xiii. SWITCH

Switch (proměnná)

```

{
    case h1: Proved' A; break;
    case h1: Proved' B; break;
    ...
    default : Proved' C;
}

```

xiv. Příklad použití Switch

```

<HTML>
<HEAD>
<TITLE>Switch v praxi</TITLE>
</HEAD>
<BODY>

```

```

<SCRIPT LANGUAGE="JavaScript">
<!--
ovoce="Jablka";
switch (ovoce)
{
    case "Pomeranče" :
        document.write("Pomeranče prodáváme za 35 Kč/kg"); break;
    case "Jablka" :
        document.write("Jablka prodáváme za 19 Kč/kg");break;
    case "Banány" :
        document.write("Banány prodáváme za 22 Kč/kg"); break;
    case "Třešně" :
        document.write("Třešně prodáváme za 48 Kč/kg");break;
    default : document.write("Je nám líto, ale na skladě nemáme "+ovoce);
}
// -->
</SCRIPT>
</BODY>
</HTML>

```

xv. Logické operátory

And &&

Or ||

Použití: expr1 || expr2

Je true pokud alespoň jeden z výrazu je true. Jinak je false.

Not !

Použití: !expr

Pro false vrací true a naopak.

Operátory porovnání

= = rovná se

!= nerovná se

> větší než

>= větší než nebo rovno

< menší než

<= menší než nebo rovno

d. Dialogová okna

V JavaScriptu je možné používat tři typy oken:

i. Výstražné informační okno

Výstražné informační okno se zobrazí příkazem `window.alert`.
`window.alert("text")`.

1. Příklad

```
<HTML>
<HEAD>
<TITLE>Ukázka vyvolání výstražného okna</TITLE>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!--
window.alert("Ahoj!\n\nJmenuji se Jan Novák a vítám vás\n na své soukromé www
stránce.");
// -->
</SCRIPT>
</BODY>
</HTML>
```



Obrázek 1 Výsledek příkazu `Window.alert`

ii. Potvrzovací okno

Potvrzovací okno se zobrazí příkazem `window.confirm`.
`A= Window.confirm(přete si skončit)`

1. Příklad

```
<HTML>
<HEAD>
<TITLE>Ukázka použití potvrzovacího okna</TITLE>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!--
listek=window.confirm("Přejete si zobrazit\n"
+ "jídelní lístek?");
```



```

if (listek)
{
    document.write("<H2>JÍDELNÍ LÍSTEK</H2>");
    document.write("0,3 l hovězí polévka<BR>");
    document.write("100 g Kung-Pao, rýže<BR>");
    document.write("ruský čaj");
}
// -->
</SCRIPT>
</BODY>
</HTML>

```

Výsledek je



Obrázek 2 Window.confirm

Poznámka

Hodnota tlačítek OK nebo Storno je buď true (OK) nebo false (Storno) v okně window.confirm a tato hodnota se uloží do proměnné listek. S touto proměnnou je nutné pak pracovat pomocí if - if (listek)...

e. Vstupní okno

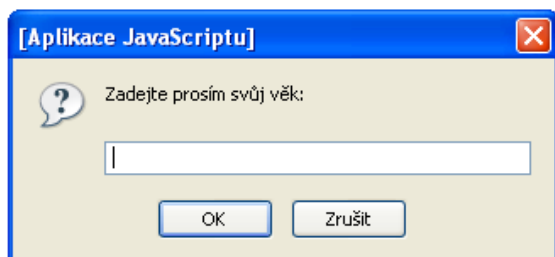
Vstupní okno se zobrazí pomocí příkazu **window.prompt** („text“, „implicitní hodnota“);
 hodnota=windows.prompt(“zadej cislo”);

1. Příklad

```

<HTML>
<HEAD>
<TITLE>Ukázka použití vstupního okna</TITLE>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!--
vek=window.prompt("Zadejte prosím svůj věk:");
vekova_kategorie = (vek>=18) ? "dospělý" : "dítě";
document.write("Máte "+vek+" rok(ů), <BR>");
document.write("patříte do kategorie "+vekova_kategorie);
// -->
</SCRIPT>
</BODY>
</HTML>

```



Obrázek 3 Window.prompt

f. Funkce v JavaScriptu

V JavaScriptu je možné definovat funkce. Syntaxe je:

function název(parametry)

Příklad funkce, kde se po zavolání funkce ramecek(barva) vytvoří kolem textu barevný rámeček.

```
{
Tělo funkce
}
Funkce s parametrem
Příklad
<HTML>
<HEAD>
<TITLE>Text vypsáný mezi rámečky (2)</TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!--
function ramecek(barva)
{
    document.write("<FONT COLOR='"+barva+"'>");
    document.write("/-----\\n");
    document.write("</FONT>");
}
// -->
</SCRIPT>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!--
document.write("<pre>");
ramecek("blue");
document.write("Text vypsáný mezi rámečky (2)\\n");
ramecek("green");
document.write("</pre>");
// -->
</SCRIPT>
</BODY>
</HTML>
```

Výsledek pak je

```
/-----\  
Text vypsaný mezi rámečky (2)  
/-----\  

```

i. Funkce – vracení hodnoty

Z funkce můžeme získat hodnotu pomocí příkazu **return**.

1. Příklad získání hodnoty z funkce.

```
<HTML>  
<HEAD>  
<TITLE>Text vypsaný mezi rámečky (2)</TITLE>  
<SCRIPT LANGUAGE="JavaScript">  
<!--  
    function mocnina(cislo) {  
        return cislo*cislo; }  
// -->  
</SCRIPT>  
</HEAD>  
<BODY>  
<SCRIPT LANGUAGE="JavaScript">  
<!--  
document.write("5^2="+mocnina(5));  
// -->  
</SCRIPT>
```

ii. Cykly

Do ... while

Syntaxe

Do

{

tělo cyklu

}

while (výraz)

Cyklus se provádí, dokud platí zadaná podmínka. Provede se vždy minimálně jednou.

1. Příklad cyklu do while, který napočítá do deseti.

```
<HTML>  
<HEAD>  
<TITLE>Zobrazení čísel od 1 do 10 </TITLE>  
</HEAD>  
<BODY>  
<SCRIPT LANGUAGE="JavaScript">  
<!--  
cislo=1;  
do  
{  
    document.write(cislo, " ");  
    cislo++;  
}
```

```

    while (cislo<11);
// -->
</SCRIPT>
</BODY>
</HTML>

```

g. Cyklus while

Syntaxe

while (výraz)

```

{
  tělo cyklu
}

```

Tělo se provádí, pokud je výraz pravdivý

1. Příklad cyklu while, který napočítá do deseti.

```

<HTML>
<HEAD>
<TITLE>Zobrazení čísel od 1 do 10 </TITLE>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!--
    cislo=1;
    while (cislo<11)
    {
        document.write(cislo+"<BR> ");
        cislo++;
    }

// -->
</SCRIPT>
</BODY>
</HTML>

```

h. Cyklus FOR

Syntaxe

For (a=2 ; a<=rozsah ; a+=2) document.write(a+" ");

1. Příklad cyklu for, který zobrazí sudá čísla od 2 do deseti.

```

<HTML>
<HEAD>
<TITLE>Zobrazovač sudých a lichých čísel</TITLE>
</HEAD>
<BODY>

```

```

<SCRIPT LANGUAGE="JavaScript">
<!--
    for (a=2;a<=10;a+=2) document.write(a+" ");
// -->
</SCRIPT>
</BODY>
</HTML>

```

i. BREAK

Pokud je tento příkaz umístěn do cyklu, lze docílit předčasného ukončení cyklu.

1. Příklad vyskočení z cyklu pomocí příkazu break

```

<HTML>
<HEAD>
<TITLE>Ukázka vyskočení z cyklu</TITLE>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!--
    jmena=new Array();
    a=0;
    window.alert("ZADÁVÁNÍ JMEN\npokud budeš chtít skončit"+
    ", zadej jako jméno slovo konec");
    while (a>=0)
    {
        jmeno=window.prompt("Zadej jméno","");
        if (jmeno=="konec") break;
        jmena[a]=jmeno;
        a++;
    }
    document.write("<H3>Zadal jsi tato jména:</H3>");
    for (b=0;b<a;b++) document.write(jmena[b]+"<BR>");
// -->
</SCRIPT>
</BODY>
</HTML>

```

j. CONTINUE

Pokud je tento příkaz umístěn do cyklu, lze docílit přeskočení následujících příkazů v cyklu a cyklus bude pokračovat dále v dalším opakování.

1. Příklad vyskočení z cyklu pomocí continue

```

<HTML>
<HEAD>
<TITLE>Ukázka vyskočení z cyklu (2)</TITLE>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">

```

```
<!--
jmena=new Array();
a=0;
window.alert("ZADÁVÁNÍ JMEN\npokud budeš chtít skončit"+
", zadej jako jméno slovo konec");
while (a>=0)
{
    jmeno=window.prompt("Zadej jméno","");
    if (!jmeno) continue;
    if (jmeno=="konec") break;
    jmena[a]=jmeno;
    a++;
}
document.write("<H3>Zadal si tyto jména:</H3>");
for (b=0;b<a;b++) document.write(jmena[b]+"<BR>");
// -->
</SCRIPT>
</BODY>
</HTML>
```

Přehled uživatelských událostí

Události okna a dokumentu:		
Událost	Popis	Lze použít na
onLoad	při úplném načtení stránky	body, img
onUnload	při opouštění stránky	body
onResize	při změně velikosti okna	body
onScroll	při skrolování, posouvání obsahu	body, textarea, cokoli s overflow

```
<body onload="alert('Dokument je načten')">
```

Události myši		
Událost	Popis	Lze použít na
onClick	při kliknutí na prvek nebo při přednastavené akci	všechny elementy, v 4. verzích prohlížečů ale jenom některé
onDbClick	při dvojkliku na prvku	
onMouseOver	při najetí myši na prvek	
onMouseOut	při odjetí z prvku	
onMouseMove	při pohybu myši nad prvkem	
onMouseDown	při stisknutí tlačítka nad prvkem	
onMouseUp	při uvolnění tlačítka nad prvkem	

Události klávesnice		
Událost	Popis	Lze použít na
onKeyPress	při stisknutí klapky ve chvíli, kdy je element aktivní	asi cokoliv
onKeyDown	při stlačení klapky ve chvíli, kdy je element aktivní	
onKeyUp	při uvolnění klapky ve chvíli, kdy je element aktivní	

Události formuláře a formulářových polí		
Událost	Popis	Lze použít na
onSubmit	těsně před odesláním formuláře, příklad	form
onReset	při vynulování formuláře tlačítkem reset	form
onFocus	při aktivaci políčka (okna)	input, select, textarea, window, odkazy
onBlur	při deaktivaci políčka (okna)	input, select, textarea
onChange	při změně hodnoty políčka	input, select, textarea
onSelect	při vybrání textu myší	body (výběr textu), textarea, input

Vlastnosti objektu Math

E	Eulerova konstanta, přirozený logaritmus (2,71828);
LN10	matematická konstanta $\log_e 10$
LN2	matematická konstanta $\log_{10} e$
PI	pi - :3,141592653;
SQRT1_2	matematická konstanta $1/\sqrt{2}$
SQRT2	matematická konstanta $\sqrt{2}$

1.4 Metody objektu String

anchor() převádí text na hypertextový odkaz s parametrem NAME

big() převádí text na tučný - přidává tagy <big> a </big>

blink() text bude v prohlížeči blikat, přidává tagy <blink> a </blink>

bold() tučný text - přidává tagy a

charAt() vrací n-tý znak řetězce. Hodnota n nesmí být menší než nula a větší než skutečný počet znaků řetězce – 1 string.charAt(n)

charCodeAt vrací číselnou hodnotu Unicode kódu znaku

concat spojí dva anebo více řetězců

fixed() přidává k textu tagy <tt> a </tt>

fontcolor() přidává tag pro barevné zvýraznění textu. Přitom barva je uvedena jako parametr metody ve formátu RGB

fontsize() nastavuje velikost textu. Ta je uvedena jako parametr metody a může se pohybovat v rozmezí od 1 do 7

indexOf() vyhledává znak (uvedený jako její parametr) v příslušném řetězci

italics() převádí text na kurzívu, přidává tagy <i> a </i>

lastIndexOf() má stejný význam jako metoda indexOf, pouze vyhledávání začne od konce řetězce

length délka řetězce

link() převádí text na hypertextový odkaz s parametrem HREF

small() přidává k textu tagy <small> a </small>

split() rozdělí řetězec na pole řetězců

strike() přidává k textu tagy <strike> a </strike>

sub() přidává k textu tagy _a

substring() vrací podřetězec obsažený v příslušném řetězci

sup() přidává k textu tagy ^a

toLowerCase() převede všechny znaky daného řetězce na malá písmena

toUpperCase() převede všechny znaky daného řetězce na velká písmena

Vlastnosti objektu string

1.5 Vlastnosti objektu Image

- border** Celé číslo pouze ke čtení, udává v bodech šířku okraje kolem obrázku. Tato hodnota se nastavuje atributem **BORDER**.
- complete** Logická hodnota pouze ke čtení, udávající, zda je obrázek celý nahrán.
- height** Celé číslo pouze ke čtení, udávající v bodech výšku obrázku. Tato hodnota se nastavuje atributem **HEIGHT**.
- hspace** Celé číslo pouze ke čtení, udávající v bodech horizontálně množství místa navíc, přidaného k pravému a levému okraji obrázku. Tato hodnota se nastavuje atributem **HSPACE**.
- lowsrc** Řetězec ke čtení a zápisu, udávající URC náhradního obrázku pro zobrazení s nízkým rozlišením.
- name** Řetězec pouze ke čtení, definovaný HTML-atributem **NAME**, udává název obrázku. Pokud obrázek dostane prostřednictvím atributu **NAME** název, je odkaz na tento obrázek umístěn navíc kromě pozice v poli **document.images[]** také ve vlastnosti **image-name**.
- src** Řetězec ke čtení a zápisu, udávající URL obrázku, který má být zobrazen. Jeho počáteční hodnota je nastavena atributem **SRC**.
- vspace** Celé číslo pouze ke čtení, udávající v bodech vertikálně množství místa navíc, přidaného nad a pod obrázek. Tato hodnota se nastavuje atributem **VSPACE**.
- width** Celé číslo pouze ke čtení, udávající v bodech šířku obrázku. tato hodnota se nastavuje atributem **WIDTH**.
- Události ovladače**
- onabort** Vyvolán, pokud uživatel přeruší načítání obrázku.
- onerror** Vyvolán, pokud při načítání obrázku dojde k nějaké chybě.
- onload** Vyvolán, pokud je nahrávání obrázku úspěšně ukončeno.

1.6 Syntaxe objektu Image v HTML

Objekt Image je vytvořen standardní HTML značkou `<IMG>`, navíc s událostními ovladači. Některé atributy `<IMG>` budou v následující syntaxi vynechány, protože nejsou JavaScriptem použity nebo mu nejsou přístupné.

<code><IMG SRC="url"</code>	obrázek určený k zobrazení
<code>WIDTH=pixels</code>	šířka obrázku
<code>HEIGHT=pixels</code>	výška obrázku
<code>[NAME="image_name"]</code>	vlastnost název obrázku
<code>[LOWSRC="url"]</code>	náhradní obrázek s nízkým rozlišením
<code>[BORDER=pixels]</code>	šířka okraje obrázku
<code>[HSPACE=pixels]</code>	místo navíc kolem obrázku horizontálně
<code>[VSPACE=pixels]</code>	místo navíc kolem obrázku vertikálně
<code>[onLoad=handler]</code>	vyvolán po úplném natažení obrázku
<code>[onError=handler]</code>	vyvolán při chybě během natahování obrázku
<code>[onAbort=handler]</code>	vyvolán při přerušení natahování uživatelem
<code>></code>	

b. window.open

window.open("URL","název_okna","vlastnosti_okna");

URL – adresa stránky, která se má nahrát do okna. Pokud nevložíme adresu a necháme prázdné uvozovky, otevře se prázdné okno.

název_okna – charakterizuje název okna, které právě otevíráme

vlastnost_okna – definují vzhled otevíraného okna. Zadáávají se vedle sebe (vlastnost=hodnota) a jsou odděleny čárkou.

tabulka 1 Vlastnosti window.open

Vlastnost	Hodnota	Funkce
toolbar	yes/no	zobrazit/nezobrazit vrchní lištu s ikonami (toolbar)
location	yes/no	zobrazit/nezobrazit řádek s adresou stránky
directories	yes/no	zobrazit/nezobrazit políčka s vybranými adresami
status	yes/no	zobrazit/nezobrazit stavový řádek
menubar	yes/no	zobrazit/nezobrazit hlavní menu prohlížeče
scrollbars	yes/no	zobrazit/nezobrazit posuvné šipky
resizable	yes/no	povolit/zakázat možnost měnit velikost okna
copyhistory	yes/no	přenést/nepřenést historii navštívených stránek
width	číslo	šířka nového okna
height	číslo	výška nového okna
top	Číslo	x-ová souřadnice nového okna
left	Číslo	y-ová souřadnice nového okna

tabulka 2 Manipulace s okny

Funkce	Činnost
close()	zavření okna
focus()	nastavení okna do popředí
blur()	nastavení okna do pozadí
moveTo(x,y)	přesun okna na souřadnice x,y
moveBy(x,y)	přesun okna o x-pixelů doprava [doleva] a y-pixelů dolů [nahoru]
resizeTo(x,y)	změna rozměrů okna na rozměr x*y pixelů
resizeBy(x,y)	změna rozměrů okna o x-pixelů doprava [doleva] a y-pixelů dolů [nahoru]
scrollTo(x,y)	rolování textu v okně na souřadnice x,y
scrollBy(x,y)	rolování textu o x-pixelů doprava [doleva] a y-pixelů dolů [nahoru]

ii. Příklad otevření fotografie v novém okně

```
<html><head>
<title>Fotoalbum</title>
<script>
<!--
function otevri_okno()
{
// vytvoření nového okna s přesnými rozměry a umístěním.
stránka fotka.html je obyčejná stránky html s vloženým obrázkem
moje=window.open("fotka.html","okno","width=386,height=257,top=10,left=200");
// moje je ukazatel na nové okno
}
// -->
</script>
</head>
```

```

<body>
<a href="#" onclick="otevri_okno();">
</a>
// zavře nové okno
<a href="#" onclick="moje.close();">zavři</A>
</body></html>

```

iii. Vlastnosti objektu window

closed	specifikuje zda bylo dané okno zavřeno nebo ne
defaultStatus	implicitní text ve stavovém řádku
length	počet rámců v okně
name	název okna
offscreenBuffering	specifikuje, zda aktualizace okna jsou vykonávány mimo obrazovku
opner	okno, které otevřelo aktuální okno
parent	rodič rámců (okno, které obsahuje rámce)
self	aktuální okno
status	text v stavovém řádku
top	synonymum nejvrchnějšího okna prohlížeče

a. Formuláře

Přístup k formuláři prostřednictvím pořadových čísel

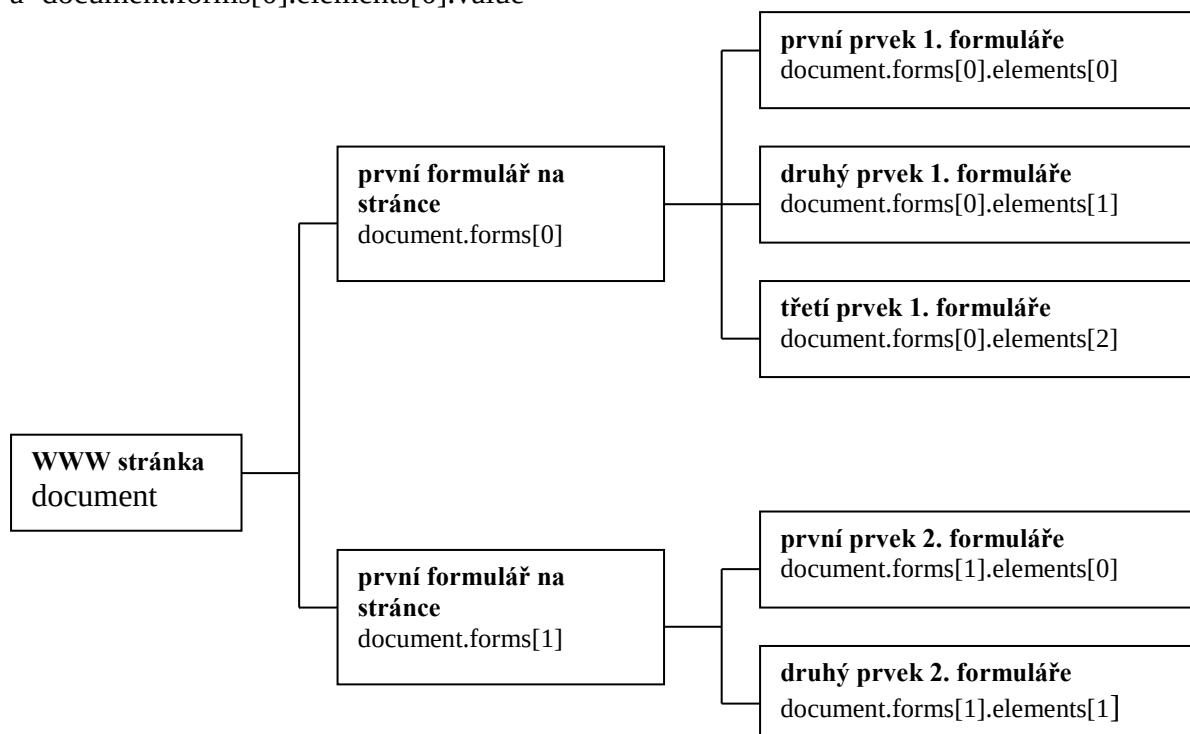
K formulářovému prvku okno přistupujeme dvěma způsoby:

Pomocí názvů

`a=document.pozdrav.okno.value;`

Pomocí indexů elementů například

`a=document.forms[0].elements[0].value`



Obrázek 4 Přístup k prvku formuláře

iv. Příklad přístupu prvku k formuláři pomocí názvů objektů

```
<HTML>
<HEAD>
<TITLE>Práce s formulářem</TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!--
function zobraz()
{
  a=document.pozdrav.okno.value;
  document.pozdrav.celkem.value=a
}
// -->
</SCRIPT>
</HEAD>
<BODY>
<FORM name="pozdrav">
zadej <INPUT TYPE="text" name="okno"><BR>
zobrazeni <INPUT TYPE="text" name="celkem">
  <INPUT TYPE="button" VALUE="Zobraz" onClick="zobraz();">
</FORM>
</BODY>
</HTML>
```

v. Příklad přístupu prvku k formuláři pomocí indexů objektů

```
<HTML>
<HEAD>
<TITLE>Práce s formulářem</TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!--
function zobraz()
{
  a=document.forms[0].elements[0].value;
  document. forms[0].elements[1].value=a
}
// -->
</SCRIPT>
</HEAD>
<BODY>
<FORM name="pozdrav">
zadej <INPUT TYPE="text" name="okno"><BR>
zobrazeni <INPUT TYPE="text" name="celkem">
  <INPUT TYPE="button" VALUE="Zobraz" onClick="zobraz();">
</FORM>
</BODY>
</HTML>
```

vi. Objekt Form

Je vstupní formulář HTML.

Vlastnosti objektu Form

Form.action – řetězcová vlastnost, ke čtení i zápisu. Udává URL, kam budou odeslány data formuláře. Počáteční hodnota této vlastnosti je určena atributem ACTION.

Form.elements[] – vstupní prvky formuláře

Přehled

form.elements[i]

form.elements.length

Form.elements je pole formulářových vstupních objektů ve formuláři. Toto pole má elements.length položek. Tyto položky mohou být kteréhokoli typu vstupního formulářového prvku: Button, Checkbox, Hidden, Password, radio, Reset, Select, Submit, Text a Textarea. Tyto formulářové vstupní objekty jsou v poli seřazeny v tomtéž pořadí, v jakém se objevují ve zdrojovém kódu HTML pro formulář.

Form.encoding kódování formulářových dat.

Form.method metoda odesílání formuláře. Jsou to buď GET nebo POST.

Ovladač Form.onreset je událost, která je vyvolána při resetu formuláře.

Ovladač Form.onsubmit() je událost, která je vyvolána při odesílání formuláře.

Form.reset() metoda resetuje prvky formuláře.

Metoda Form.submit() odeslání formuláře

Vlastnost Form.target okno pro výsledky formuláře

Vlastnosti

checked logická hodnota, určující stav formulářových prvků – zatržen (checked), nebo nezatržen (unchecked)

defaultChecked logická hodnota, určující počáteční stav formulářových prvků. Tato hodnota je dána atributem CHECKED.

defaultValue implicitní hodnota formulářového prvku

form odkaz na objekt Form, obsahující tento prvek

length u formulářového prvku Select udává vlastnost počet možností nebo voleb obsažených v poli options[] tohoto prvku.

name název prvku

options[] u formulářového prvku Select obsahuje toto pole objekty Option, představující možnosti nebo volby, zobrazené objektem Select

selectedIndex počet zobrazených možností prvku Select.

type typ formulářového prvku.

value hodnota prvku, vyslanou serveru při odsílání formuláře

Metody

blur() odstraní zaměření prvku

click() simuluje klepnutí myši na formulářový prvek

focus() umístí zaměření na daný prvek

select() vybere text, který se objeví v prvku (např. u text)

Událostní ovladače

Onblur Událost je vyvolána v okamžiku, kdy element přestává být aktivní.

OnChange Událost je vyvolána, pokud se změnila hodnota vstupního pole formuláře.

OnClick Událost je vyvolána po kliknutí myši na element.

OnFocus Událost je vyvolána v okamžiku, kdy je element aktivován myší nebo pomocí tabulátoru.

Prvek button

Syntaxe HTML

<FORM>

<INPUT

TYPE="button" definuje, že toto je tlačítko

VALUE="label"	text, který se objeví na tlačítku
[NAME="name"]	jméno může být později použito k odkazu na toto tlačítko
[onClick="handler"]	příkazy JavaScriptu, vykonané v případě, že je na tlačítko klepnuto

>

</FORM>

vii. Prvek radio

Prvek **Radio** se používá vždy ve skupině vzájemně se vylučujících voleb s týmž názvem. Odkaz na jednotlivý prvek RADIO ve skupině používá tuto syntaxi:

```
form.radio_name[j]
form.radio_name.length
form.elements['name'][j]
form.elements['name'].length
```

Metody

blur()	Odstraní zaměření z tohoto tlačítka radio.
click()	Simuluje klepnutí na tlačítko radio.
focus()	Umístí zaměření na tlačítko radio.

Události ovladače

onblur()	Je vyvolán, když tlačítko radio ztratí zaměření.
onclick()	Je vyvolán, když se na tlačítko radio klepne.
onfocus()	Je vyvolán, když tlačítka radio získá zaměření.

Syntaxe HTML

```
<FORM>
<INPUT
    TYPE="radio"                Definuje, že toto je tlačítka radio.
    [ NAME="name"]             Jméno může být později použito k odkazu na toto tlačítko.
    [VALUE = "value"]          Hodnota vrácená, pokud je tlačítko zatrženo.
    [CHECKED]                  Určuje je-li políčko na počátku prvotně stlačeno.
    [onClick="handler"]        Příkazy JavaScriptu, vykonané v případě, že je na tlačítko
klepnuto.
>
```

label	Text HTML, který se objeví vedle tlačítka
-------	---

</FORM>

Vlastnosti

Radio.checked	Pokud je RADIO vybráno – true, jinak false.
Radio.click	Simuluje klepnutí myši na formulářovém prvku.
Radio.defaultChecked	Počáteční stav tlačítka.
Radio.focus	Umístí zaměření kurzoru na formulářový prvek.
Radio.form	Formulář, obsahující daný prvek.
Radio.name	Jméno prvku.
Radio.onblur	Je vyvolán, ztratí-li formulářový prvek zaměření.
Radio.click	Je vyvolán, je-li tlačítko Radio vybráno.
Radio.onfocus()	Je vyvolán, získá-li formulářový prvek zaměření.
Radio.type	Typ formulářového prvku.
Radio.value	Hodnota, vrácená při odeslání formuláře.

viii. Prvek checkbox – grafické zatrhávací políčko

Jednoduchý prvek Checkbox s jedinečným jménem může být odkazován jedním z následujících způsobů:

form.checkbox_name

form.elements[i]

Pokud formulář obsahuje skupinu zatrhávacích políček s týmž jménem, mohou být umístěny z poli a lze se na ně odkazovat následovně:

form.checkbox_name[j]

form.checkbox_name.length

form.elements['name'][j]

form.elements['name'].length

Vlastnosti prvku checkbox

checked Logická hodnota (pro čtení i zápis), zda je políčko zatrhnuto.

defaultChecked Logická hodnota (jen ke čtení) určující počáteční stav políčka. Může být definován HTML-atributem CHECKED.

form Odkaz (jen ke čtení) na objekt Form, obsahující Checkbox.

name Řetězec (jen ke čtení), nastavený HTML-atributem NAME, určující jméno Checkboxu.

type Řetězec (jen ke čtení), určující typ tohoto formulářového prvku. Pro prvky Checkbox má tento řetězec hodnotu „checkbox“.

value Řetězec (pro čtení a zápis), prvotně nastaven HTML-atributem VALUE.

Metody prvku checkbox

blur() Odstraní zaměření z tohoto políčka.

click() Simuluje klepnutí na políčko.

focus() Umístí zaměření na políčko.

Události ovladače

onblur() Je vyvolán, když políčko ztratí zaměření.

onclick() Je vyvolán, když se na políčko klepne.

onfocus() Je vyvolán, když políčko získá zaměření.

Syntaxe HTML

<FORM>

<INPUT

TYPE="checkbox" definuje, že toto je zatrhávací políčko

[NAME="name"] jméno může být později použito k odkazu na toto políčko

[VALUE = "value"] hodnota vrácená, pokud je políčko zatrženo

[CHECKED] určuje je-li políčko prvotně zatrženo

[onClick="handler"] příkazy JavaScriptu, vykonané v případě, že je na políčko klepnuto.

>

</FORM>

Výpis všech elementů formuláře cyklem for

<SCRIPT LANGUAGE="JavaScript">

<!--

function show(obj, obj_name)

{

var result = "";

for (var i in obj)

result +=obj_name + "."+i+"="+obj[i]+"
"

return result

}

document.Brite(show(document.zobrazoval.elements, "document.Zobrazoval..elements"));

ix. Kontrola formuláře před odesláním pomocí JavaScriptu.

Po odeslání formuláře pomocí tlačítka Submit událost OnSubmit zavolá kontrolní funkci. V případě návratu hodnoty false z kontrolní funkce se formulář neodešle.

Kontrola formuláře před odesláním pomocí JavaScriptu.

Po odeslání formuláře pomocí tlačítka Submit událost OnSubmit zavolá kontrolní funkci. V případě návratu hodnoty false z kontrolní funkce se formulář neodešle.

x. Příklad kontroly na neprázdnost okna

```
<HTML>
<HEAD>
<TITLE>Kontrola zadané e-mailové adresy</TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!--
function kontrola()
{
  if (document.prvni.jmeno.value=="")
  {
    window.alert("nezadal jsi jméno, opakuj zadání");

  }
  else   window.alert("správně zadané");

}
// -->
</SCRIPT>
</HEAD>
<BODY>
<FORM NAME="prvni" >
jméno<INPUT TYPE="text" NAME="jmeno" >
<INPUT TYPE="button" VALUE="kontrola" onClick="kontrola();">
</FORM>
</BODY>
</HTML>
```

Poznámka - metoda indexOf

Vrací index znaku v rámci objektu string, který první odpovídá specifikované hodnotě, a hledání začíná na *fromIndex*.

jmeno_string.indexOf(hodnota, [fromIndex])

jmeno_string je libovolný řetězec nebo vlastnost existujícího objektu.

hodnota je řetězec nebo vlastnost existujícího objektu, reprezentující hodnotu, která se má hledat.

fromIndex je místo, z kterého se začne hledat. Může to být libovolné celé číslo mezi 0 a **jmeno_string.length - 1**, nebo vlastnost existujícího objektu.

Znaky v řetězci jsou indexovány zleva doprava. Index prvního znaku je 0 a index posledního znaku je `jmeno_string.length - 1`.

Kontrola na délku

```
<HTML>
<HEAD>
<TITLE>Práce s formulářem</TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!--
function zobraz()
{
  a=document.pozdrav.okno.value;

  document.pozdrav.celkem.value=a.length;
}
// -->
</SCRIPT>
</HEAD>
<BODY>
<FORM name="pozdrav">
zadej <INPUT TYPE="text" name="okno"><BR>

delka je <INPUT TYPE="text" name="celkem">

  <INPUT TYPE="button" VALUE="Zobraz" onClick="zobraz();">
</FORM>
</BODY>
</HTML>
```

Příklad – kontrola na přítomnost zavináče v e-mailu

```
<HTML>
<HEAD>
<TITLE>Kontrola zadané e-manilové adresy</TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!--
function kontrola(adresa)
{
adresa=document.f1.email.value;
if (adresa.indexOf("@")==-1)
{
document.f1.Info.value="Zadal jsi NESPRÁVNOU "+
"e-manilovou adresu! Zadej ji znovu!";
return false;
}
}
// -->
</SCRIPT>
</HEAD>
<BODY>
<FORM NAME="f1" onSubmit="return kontrola();">
<INPUT TYPE="text" NAME="Info" SIZE="55"
VALUE="Zadej svoji e-mailovou adresu a klikni na Odešli!">
<P>E-mail:
<INPUT TYPE="text" NAME="email" SIZE="25" VALUE="">
<INPUT TYPE="submit" VALUE="Odešli">
</FORM>
</BODY>
</HTML>
```

i. Regulární výrazy

tabulka 3 Příklady regulárních výrazů

Výraz	Znamená	Vyhovují	Nevyhovují
Abc	Řetězec, který obsahuje tento podřetězec	abc, aascvabcmn,abcd	abbbvvvc, cba,
^	Začátek řetězce		
^abc		abcd,abc,	bbabc, nabc
\.	tečka		

\\	lomítko		
*	výraz se může opakovat, nebo nemusí být		
a*b		aaaabbbb,ab,agb	aaaadddd, a
+	výraz se musí objevit alespoň jednou		
.	libovolný znak		
?	výraz není, nebo je jedenkrát		
\$	konec řetězce	hasghaa,a	aafdgd, cc, an
a\$			
a{2,6}	minimální a maximální počet opakování a	aa,aaa,aaaa,aaaaa,aaaaaa	a,aaaaaaa
a{5,}	a je minimálně 5 krát		
a{5}	a je maximálně 5 krát		
[abcd]	jeden znak z množiny		
[a-d]	zkrácený výpis předchozího řádku		
[a-z]	malé písmena		
[A-Z]	velké písmeno		
[a-zA-Z]	malé nebo velké písmeno		
[^ab]	regulární výraz vyhoví libovolnému znaku, kromě těch v závorce		

ii. Jednoduchý program pro testování regulárního výrazu

```

<HTML>
<HEAD>
<TITLE>Kontrola zadané e-mailové adresy</TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!--

function kontrola()
{
    re = new RegExp("^[^@]+@[^.]+\..+$");
    if (re.test(f1.neco.value))
        {alert("platí");}
    else
        alert("neplatí")
}
// -->
</SCRIPT>
</HEAD>
<BODY>
Zadej řetězec a já provedu test na regulární výraz<BR><BR>
<FORM NAME="f1" action="neco.php">
<P>Zadej řetězec:
<INPUT TYPE="text" NAME="neco" SIZE="25" >
<INPUT TYPE="button" VALUE="Odešli" onClick= "kontrola();">
</FORM>
</BODY>
</HTML>

```

Výsledek tohoto skriptu pak vypadá následovně

Zadej řetězec a já provedu test na regulární výraz

Zadej řetězec:



iii. Příklad použití regulárního výrazu

Kontroluje se, zda je výraz číslo.

function kontrola()

{

re = new RegExp("[1234567890]");

if (!re.test(document.F1.Email.value))

{alert("Chyba - zadaný řetězec obsahuje nečíselné znaky");

formular.Email.focus();

}

}

// -->

</SCRIPT>

</HEAD>

<BODY>

<FORM METHOD=POST name="F1" onSubmit="kontrola();" >

<INPUT TYPE=TEXT NAME="Email">

<INPUT TYPE=SUBMIT VALUE=" Registrace ">

</FORM>

</BODY>

</HTML>

Příklad regulárního výrazu kontroly e-mailu: `^[^@]+@[^\.]+\.[[:alnum:]]{1,3}$`

iv. Funkce escape()

Zakódování řetězce pro přenos. Vytvoří se nový řetězec, kde všechny mezery, interpunkce, diakritika a všechny ostatní znaky, které nejsou ASCII-písmeny nebo číslicemi, jsou převedeny do formy %DX, kde DX jsou dvě šestnáctkové číslice, reprezentující kódování znaku dle ISO-8859-1 (LATIN-1). Například znak ! má kód 33. K dekodování se používá unescape().

1.7 Metody objektu navigator

javaenabled vrací true/false podle toho, zda je aktivovaná podpora jazyka Java

Vlastnosti objektu navigator

appName kódový název prohlížeče stránek

appMinorVersions subverze prohlížeče stránek

appName	název prohlížeče stránek
appVersion	verze prohlížeče stránek
browserLanguage	jazyk prohlížeče stránek
cookieEnabled	hodnota true/false podle toho, zda je povoleno uchovávání cookies
cpuClass	procesorová třída počítače, na kterém běží prohlížeč
onLine	hodnota true/false podle toho, zda je prohlížeč v režimu online/offline
platform	název operačního systému počítače, na kterém běží prohlížeč
systemLanguage	implicitně nastavený jazyk operačního systému
userAgent	otisk prohlížeče (řetězcový ekvivalent HTTP hlavičky user-agent)
userLanguage	přirozený jazyk operačního systému
Vlastnosti objektu screen	
availHeight	výška pracovní plochy obrazovky operačního systému
availWidth	šířka pracovní plochy obrazovky operačního systému v pixelech
colorDepth	barevná hloubka (počet bitů připadajících na barvu)
fontSmoothingEnabled	informuje o tom, zda je aktivováno vyhlazování fontů
height	vertikální rozlišení obrazovky v pixelech
width	horizontální rozlišení obrazovky v pixelech

Datum a čas

Pro práci s datem a časem je potřeba nejprve vytvořit nový objekt typu Date. Pak je možné z tohoto **nového objektu** získávat jednotlivé informace o času.

```
cas=new Date();
```

Příkazem new vytvoříme nový objekt cas, který má stejné vlastnosti jako objekt Date().

Pak již lze pomocí metod a vlastností přistupovat k novému objektu **cas**.

Metody objektu Date

getDate()	Vrací číslo dne uloženého v instanci objektu Date v tomto měsíci.
getDay()	Vrací pořadové číslo dne v týdnu uloženém v instanci objektu Date. Číslo je v rozmezí od 0 (neděle) do 6 (sobota)
getFullYear()	vrací položku rok v čtyřciferném tvaru
getHours()	vrací číslo v rozmezí 0 (půlnoc) až 23, které vyjadřuje hodinu právě uloženou v instanci objektu Date
getMilliseconds()	vrací položku milisekundy
getMinutes()	vrací číslo v rozmezí 0 až 59, které vyjadřuje minutu právě uloženou v instanci objektu Date
getMonth()	vrací číslo v rozmezí 0 (leden) až 11 (prosinec), které vyjadřuje měsíc právě uložený v instanci objektu Date
getSeconds()	vrací číslo v rozmezí 0 až 59, které vyjadřuje sekundu právě uloženou v instanci objektu Date
getTime()	vrací číslo, které vyjadřuje počet milisekund, které uplynuly od 1. 1. 1970 do data uloženého právě v instanci objektu Date. Může sloužit při porovnávání dvou dat
getTimezoneOffset()	vrací rozdíl (v minutách) mezi datem uloženým v instanci objektu Date a GMT (časem, který je právě teď v Grennwiche)
getYear()	vrací rok ve tvaru RR (97)
setDate(x)	nastavení dne v měsíci pro instanci objektu Date, kde x je hodnota v rozmezí 1 až 31

setDay	nastaví den v měsíci
SetFullYear	vrací položku rok v čtyřciferném tvaru
setHours(x)	nastavení hodiny pro instanci objektu Date, kde x je hodnota v rozmezí 0 (půlnoc) až 23
setMilliseconds	nastaví položku milisekundy
setMinutes(x)	nastavení minut pro instanci objektu Date, kde x je hodnota v rozmezí 0 až 59
setMonth	nastaví položku měsíc
setSeconds	nastaví položku sekundy
setTme(x)	nastavení doby, která uplynula od 1.1.1970 pro instanci objektu Date, kde x je číslo vyjadřující počet milisekund
setYear(x)	nastavení roku pro aktuální instanci objektu Date, kde x je hodnota vyjadřující rok aktuálního letopočtu
toGMTString()	převádí čas uložený v instanci objektu Date na čas používající formát GMT
toString()	vrací řetězec reprezentující objekt data
toLocaleString()	převádí čas uložený v instanci objektu Date na lokální čas. Je tedy metodou přesně opačnou k metodě předchozí
UTC()	převádí datum a čas (v lokálním formátu) na počet milisekund od 1.1.1970
parse()	převádí datum a čas (ve formátu GMT) na počet milisekund od 1.1.1970

Příklad 1

```

<HTML>
<HEAD>
<TITLE>Výpis aktuálního datumu a času</TITLE>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!--
    dnes=new Date();
    document.write(dnes);
    document.write("<BR>");
    dne=dnes.getMonth();
    document.write(dne);

// -->
</SCRIPT>
</BODY>
</HTML>

```

Vypíše se

Příklad 2

```
<HTML>
<HEAD>
<TITLE>Výpis aktuálního datumu a času</TITLE>
</HEAD>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
<!--
  dnes=new Date();
  den=dnes.getDate();
  mesic=dnes.getMonth()+1;
  rok=dnes.getFullYear();
  document.write(den+"."+mesic+"."+rok);
// -->
</SCRIPT>
</BODY>
</HTML>
```

Vypíše 22.4.2008

Načasování v JavaScriptu

K načasování slouží příkaz **window.setTimeout(„co se stane“,čas v ms)**

Příklad

```
<HTML>
<HEAD>
<TITLE>Načasování v JavaScriptu</TITLE>
<SCRIPT LANGUAGE="JavaScript">
<!--
function obnov()
{
  window.setTimeout("window.location='soubor.htm'",3000);
}
// -->
</SCRIPT>
</HEAD>
<BODY onLoad="obnov();">
```

Každé 3 sekundy se tato stránka znovu nahraje

</BODY>

</HTML>

Poznámka

window.location – tato funkce objektu window nahraje do okna stránku.

window.clearTimeout(ukazatel na window.setTimeout) – zruší window.setTimeout

Příklad

<HTML>

<HEAD>

<TITLE>Hodinky</TITLE>

<SCRIPT LANGUAGE="JavaScript">

<!--

zapnute_hodinky=false;

function zapni_hodinky()

{

 dnes=new Date(); vytvoří nový objekt typu Date s názvem
dnes

 with (dnes) {

 hodina=getHours();

minuta=getMinutes();

sekunda=getSeconds();

}

cas=hodina;

cas+=((minuta<10) ? ":0" : ":")+minuta;

cas+=((sekunda<10) ? ":0" : ":")+sekunda;

dokument.write(cas); napíše se na obrazovku čas

ukazatel=window.setTimeout("zapni_hodinky()",1000);

}

function vypni_hodinky()

{

 if (zapnute_hodinky)

 {

 window.clearTimeout(ukazatel);

 window.status+=" (zobrazování času bylo pozastaveno)";

```
    zapnute_hodinky=false;  
}
```

```
// -->
```

```
</SCRIPT>
```

```
</HEAD>
```

```
<BODY>
```

```
<FORM>
```

```
<INPUT TYPE="button" onClick="zapni_hodinky();" VALUE="Zapni hodinky">
```

```
<INPUT TYPE="button" onClick="vypni_hodinky();" VALUE="Vypni hodinky">
```

```
</FORM>
```

```
</BODY>
```

```
</HTML>
```